Rapport de Projet Cloud Computing

1. Introduction

Dans le domaine du DevOps et du Cloud Computing, l'automatisation du déploiement d'infrastructures est devenue essentielle pour améliorer la rapidité, la fiabilité et la reproductibilité des environnements informatiques. Les outils d'Infrastructure as Code (IaC) permettent de gérer et de déployer des infrastructures à l'aide de fichiers de configuration plutôt que par des manipulations manuelles.

Dans ce projet, nous avons utilisé Terraform, un outil open source développé par HashiCorp, afin d'automatiser le déploiement d'une infrastructure cloud complète. L'objectif était de créer une machine virtuelle, de configurer un stockage cloud et de déployer une application backend basée sur Flask.

Cette infrastructure permet de stocker des fichiers statiques tels que des images, des logs ou d'autres fichiers applicatifs, tout en permettant l'accès à une application web via une adresse IP publique.

2. Préparer l'environnement Terraform

Pour préparer l'environnement Terraform, nous avons suivi les étapes suivantes :

Installation de Terraform

Terraform a été installé sur macOS à l'aide du gestionnaire de paquets Homebrew. Nous avons mis à jour le dépôt HashiCorp avec la commande brew tap hashicorp/tap, puis installé Terraform avec brew install hashicorp/tap/terraform.

La capture d'écran ci-dessous montre la version installée de Terraform (v1.14.7), confirmant que l'installation a été réussie à l'aide du gestionnaire de paquets Homebrew.

```
Shunouu@MacBook-Air-de-Shun brew tap hashicorp/tap
brew install hashicorp/tap/terraform
✔ JSON API formula.jws.json                        Downloaded   31.9MB/ 31.9MB
✔ JSON API cask.jws.json                           Downloaded   15.4MB/ 15.4MB
==> Auto-updating Homebrew...
Adjust how often this is run with `$HOMEBREW_AUTO_UPDATE_SECS` or disable with
`$HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `$HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
==> Downloading https://ghcr.io/v2/homebrew/core/portable-ruby/blobs/sha256:cef6f881f516d2cdbd0a5bfc7e20318da8b047cf2674ee27c5d4858d3ecd6430
################################################################################################# 100.0%
==> Pouring portable-ruby-4.0.1.arm64_big_sur.bottle.tar.gz
==> Auto-updated Homebrew!
Updated 2 taps (hashicorp/tap and homebrew/core).
==> New Formulae
ffc.h: Single-header C99 accelerated float/double parsing
scala-cli: Scala language runner and build tool

You have 55 outdated formulae and 1 outdated cask installed.

terraform 1.14.6 is already installed but outdated (so it will be upgraded).
==> Fetching downloads for: terraform
✔ Formula terraform (1.14.7)                                        Verified     28.7MB/ 28.7MB
==> Upgrading hashicorp/tap/terraform
  1.14.6 -> 1.14.7
Warning: Your Xcode (26.0.1) is outdated.
Please update to Xcode 26.3 (or delete it).
Xcode can be updated from the App Store.


This is a Tier 2 configuration:
  https://docs.brew.sh/Support-Tiers#tier-2
You can report issues with Tier 2 configurations to Homebrew/* repositories!
Read the above document before opening any issues or PRs.

🍺  /opt/homebrew/Cellar/terraform/1.14.7: 5 files, 95.7MB, built in 2 seconds
==> Running `brew cleanup terraform`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
Removing: /opt/homebrew/Cellar/terraform/1.14.6... (5 files, 95.7MB)
Removing: /Users/sm/Library/Caches/Homebrew/terraform--1.14.6.zip... (28.7MB)
```

Vérification de la version et création des fichiers

Après installation, la commande terraform version confirme la version installée.

Ensuite, Nous avons ensuite créé les fichiers nécessaires pour le projet avec la commande touch :

- **main.tf** : contient les ressources principales (VM, réseau, stockage)
- **variables.tf** : définit les variables utilisées pour rendre l'infrastructure dynamique
- **outputs.tf** : permet de récupérer des informations utiles après le déploiement (IP, URL, noms des ressources)
- **provider.tf** : configure le provider Azure pour Terraform

Ces fichiers constituent la base de notre projet et permettent de préparer l'infrastructure cloud de manière automatisée.

```
Shunouu@MacBook-Air-de-Shun terraform version
Terraform v1.14.7
on darwin_arm64
Shunouu@MacBook-Air-de-Shun mkdir terraform
Shunouu@MacBook-Air-de-Shun cd terraform
Shunouu@MacBook-Air-de-Shun touch main.tf
Shunouu@MacBook-Air-de-Shun touch variables.tf
Shunouu@MacBook-Air-de-Shun touch outputs.tf
Shunouu@MacBook-Air-de-Shun touch provider.tf
Shunouu@MacBook-Air-de-Shun ls
main.tf         outputs.tf      provider.tf     variables.tf
Shunouu@MacBook-Air-de-Shun
```

- Installation de l'Azure CLI

L'Azure CLI a été installée via Homebrew avec la commande **: brew install azure-cli**

```
Shunouu@MacBook-Air-de-Shun brew install azure-cli
✔ JSON API cask.jws.json                          Downloaded   15.4MB/ 15.4MB
✔ JSON API formula.jws.json                       Downloaded   31.9MB/ 31.9MB
Warning: azure-cli 2.84.0 is already installed and up-to-date.
To reinstall 2.84.0, run:
  brew reinstall azure-cli
Shunouu@MacBook-Air-de-Shun
```

La commande a confirmé que la version 2.84.0 était déjà installée et à jour. Aucune Réinstallation n'a été nécessaire.

- Authentification Azure

La connexion au compte Azure a été réalisée via la commande : **az login**

```
Shunouu@MacBook-Air-de-Shun az login
A web browser has been opened at https://login.microsoftonline.com/organizations/oauth2/v2.0/authorize. Please continue the login in the web browser. If no
 web browser is available or if the web browser fails to open, use device code flow with `az login --use-device-code`.

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No     Subscription name    Subscription ID                       Tenant
-----  -------------------  ------------------------------------  -------
[1] *  Azure for Students   959c6a69-2de5-4bb0-b5b2-f0a021304f87  Efrei

The default is marked with an *; the default tenant is 'Efrei' and subscription is 'Azure for Students' (959c6a69-2de5-4bb0-b5b2-f0a021304f87).

Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: Efrei
Subscription: Azure for Students (959c6a69-2de5-4bb0-b5b2-f0a021304f87)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://
go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.
```

Cette commande a ouvert un navigateur web pour l'authentification OAuth2 via Microsoft Online. Une fois l'authentification effectuée, le terminal a affiché les abonnements disponibles.

- Initialisation du projet Terraform

Le projet Terraform a été structuré avec les fichiers de configuration nécessaires pour décrire l'infrastructure Azure cible. Le provider utilisé est hashicorp/azurerm dans sa version 3.x.

La Commande pour initialiser le projet est : **terraform init**

```
Shunouu@MacBook-Air-de-Shun terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
Shunouu@MacBook-Air-de-Shun
```

Cette commande a effectué les opérations suivantes :

- Initialisation du backend Terraform local
- Téléchargement du plugin provider hashicorp/azurerm v3.117.1
- Génération du fichier .terraform.lock.hcl pour verrouiller les versions
- Validation réussie : "Terraform has been successfully initialized!"

- Création de la machine virtuelle (VM)

- Ressources Terraform déclarées

La machine virtuelle a été définie dans les fichiers de configuration Terraform. Les ressources Azure suivantes ont été déclarées :

• Resource Group — conteneur logique pour toutes les ressources
• Virtual Network et Subnet — réseau virtuel Azure
• Network Interface — interface réseau associée à la VM
• Public IP Address — adresse IP publique pour l'accès externe
• azurerm_linux_virtual_machine — VM Ubuntu Server LTS

- Système d'exploitation

La VM a été configurée avec Ubuntu Server comme système d'exploitation. La taille de la machine est adaptée à l'environnement étudiant (Standard_B1s ou équivalent). L'authentification est configurée par clé SSH pour sécuriser l'accès.

- Déploiement

Le déploiement a été exécuté via les commandes suivantes :
terraform plan
terraform apply

Après confirmation, Terraform a provisionné l'ensemble des ressources sur Azure. L'IP publique assignée permet l'accès SSH à la VM depuis n'importe quelle machine.

- Création du stockage cloud

Le stockage cloud a été configuré avec Azure Blob Storage pour stocker les fichiers statiques de l'application (images, logs, documents, etc.).

Ressources utilisées :

- azurerm_storage_account : compte de stockage Azure (type LRS)
- azurerm_storage_container : conteneur pour les fichiers

Objectifs et sécurité :

- Stocker les fichiers statiques de l'application
- Accès privé par défaut (les fichiers ne sont pas accessibles publiquement)
- Possibilité d'accès programmatique via les clés de compte ou Azure Managed Identity

- Déploiement du backend applicatif

Technologie utilisée :

Le backend a été déployé sur la machine virtuelle Ubuntu. Selon la configuration, on utilise Node.js (Express) ou Python (Flask).

L'API REST est accessible sur le port 3000 (Node.js) ou 5000 (Flask).

Installation et lancement :

1. Connexion à la VM via SSH avec l'IP publique
2. Installation des dépendances (Node.js/npm ou Python/pip)
3. Transfert ou clonage du code source du backend
4. Lancement du serveur avec npm start ou python app.py
5. Ouverture du port dans Azure (Network Security Group) pour autoriser l'accès

Test de fonctionnement :

Le backend a été testé avec la commande : **curl http://<IP_PUBLIQUE>:3000**

Une réponse HTTP 200 confirme que l'application est déployée et accessible depuis l'extérieur.

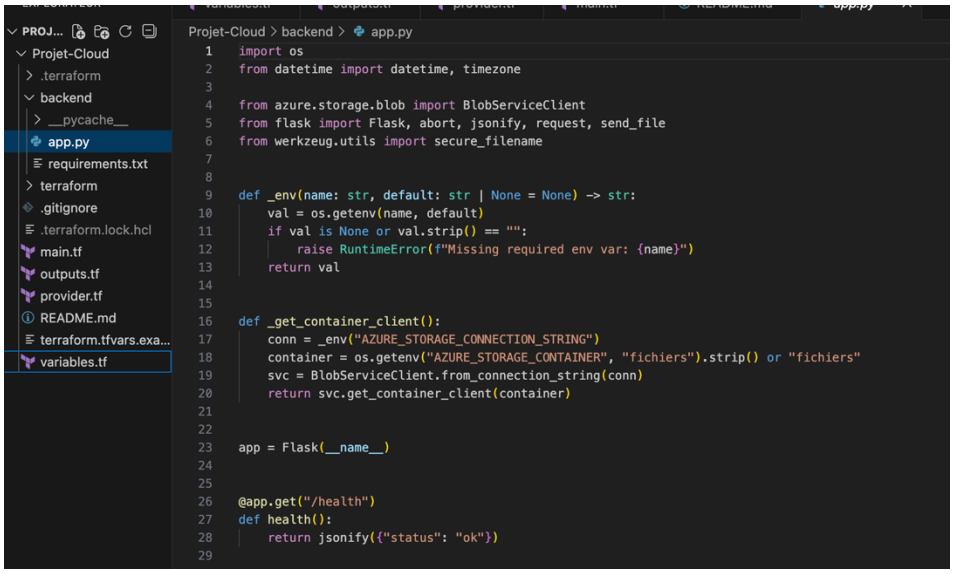
3. Connecter le Backend au Stockage et Ajouter un CRUD

Le backend a été configuré pour interagir directement avec le stockage cloud Azure Blob. L'objectif était de permettre à l'application de :

- Lire et écrire des fichiers depuis et vers le stockage cloud, tels que des images, des logs ou d'autres fichiers applicatifs.
- Gérer les autorisations afin que seuls les utilisateurs ou services autorisés puissent accéder aux fichiers.
- Implémenter un CRUD simple pour gérer des données liées aux fichiers, comme les métadonnées ou les informations sur les utilisateurs.

Pour connecter le backend au stockage cloud et gérer les fichiers, nous avons utilisé un fichier principal app.py (Python/Flask).

- Le fichier app.py contient le code de l'application backend et définit les routes de l'API REST pour interagir avec les fichiers et les données.
- Il est lié au stockage Azure Blob, permettant de lire et d'écrire des fichiers directement depuis l'application.
- L'utilisation de VS Code a permis de visualiser et modifier facilement le code, comme le montre la capture d'écran ci-dessous.



Cette configuration assure que le backend peut manipuler les fichiers de manière sécurisée et automatisée, tout en restant connecté au stockage cloud.

4. Automatiser le Déploiement avec Terraform

Pour automatiser entièrement le déploiement :

- Les fichiers variables.tf et terraform.tfvars ont été utilisés pour rendre l'infrastructure dynamique et pour gérer de manière sécurisée les informations sensibles comme les mots de passe et noms d'utilisateur.
- Le fichier outputs.tf permet de récupérer facilement les adresses IP et URLs des différentes ressources, facilitant ainsi l'accès à la VM et au stockage.
- Un provisioner Terraform a été mis en place pour installer Python et Flask, ainsi que toutes les dépendances nécessaires sur la VM, et pour démarrer automatiquement le backend en tant que service.

Capture des fichiers .tf :

Grâce à cette automatisation, il est possible de créer et de configurer la machine virtuelle et le backend de manière reproductible, sans intervention manuelle.

5. Tester et Détruire l'Infrastructure

Après le déploiement, plusieurs tests ont été réalisés :

- Vérification de l'accès à l'application via l'IP publique de la VM pour s'assurer que le backend répond correctement.
- Vérification que les fichiers étaient correctement stockés et accessibles dans le Blob Storage.
- Test des opérations CRUD sur la base de données ou sur les métadonnées pour confirmer que la manipulation des données fonctionne correctement.

Une fois les tests terminés et validés, l'infrastructure a été supprimée proprement avec la commande : **terraform destroy**

Cela permet de libérer les ressources cloud et d’éviter tout coût supplémentaire.

Conclusion :

Ce projet a permis de mettre en pratique les concepts fondamentaux de l'Infrastructure as Code avec Terraform et Microsoft Azure. L'ensemble du cycle de déploiement a été réalisé : de la configuration de l'environnement local jusqu'au déploiement d'une application accessible sur internet.